

Greedy and Dynamic Programming (Program)

Scheduling Problems

- ◆ Given a set of tasks (or, processes), our goal is to accommodate maximum of them, or maximise total profit, etc.
- ◆ Think of a server (or some kind of resource) is doing the computation.

Why Scheduling?

Computing: CPU scheduling, cloud jobs, GPU clusters, data centers

Manufacturing: machine usage, assembly lines, robotics

Logistics: delivery routing, port and airport operations

Healthcare: operating rooms, diagnostic equipment, staff shifts.

(Interval) Scheduling

- ◆ We have n requests, where each request r_i has a start time s_i and end time t_i .
- ◆ Each of these requests can be seen as an interval - $[s_i, t_i]$
- ◆ Hence, we have n intervals, $[s_1, t_1], \dots, [s_n, t_n]$.

(Interval) Scheduling

- ◆ We have n requests, where each request r_i has a start time s_i and end time t_i .
- ◆ Each of these requests can be seen as an interval - $[s_i, t_i]$
- ◆ Hence, we have n intervals, $[s_1, t_1], \dots, [s_n, t_n]$.

Goal: Maximize the total number of requests that we serve.

Interval Scheduling

- ◆ Constraints.
 - ◆ If we decide to allocate or serve a request, then we have to do it completely. No partial allocation is allowed.
 - ◆ The resource can be allocated to only one request at a time. Hence, no overlap is allowed.

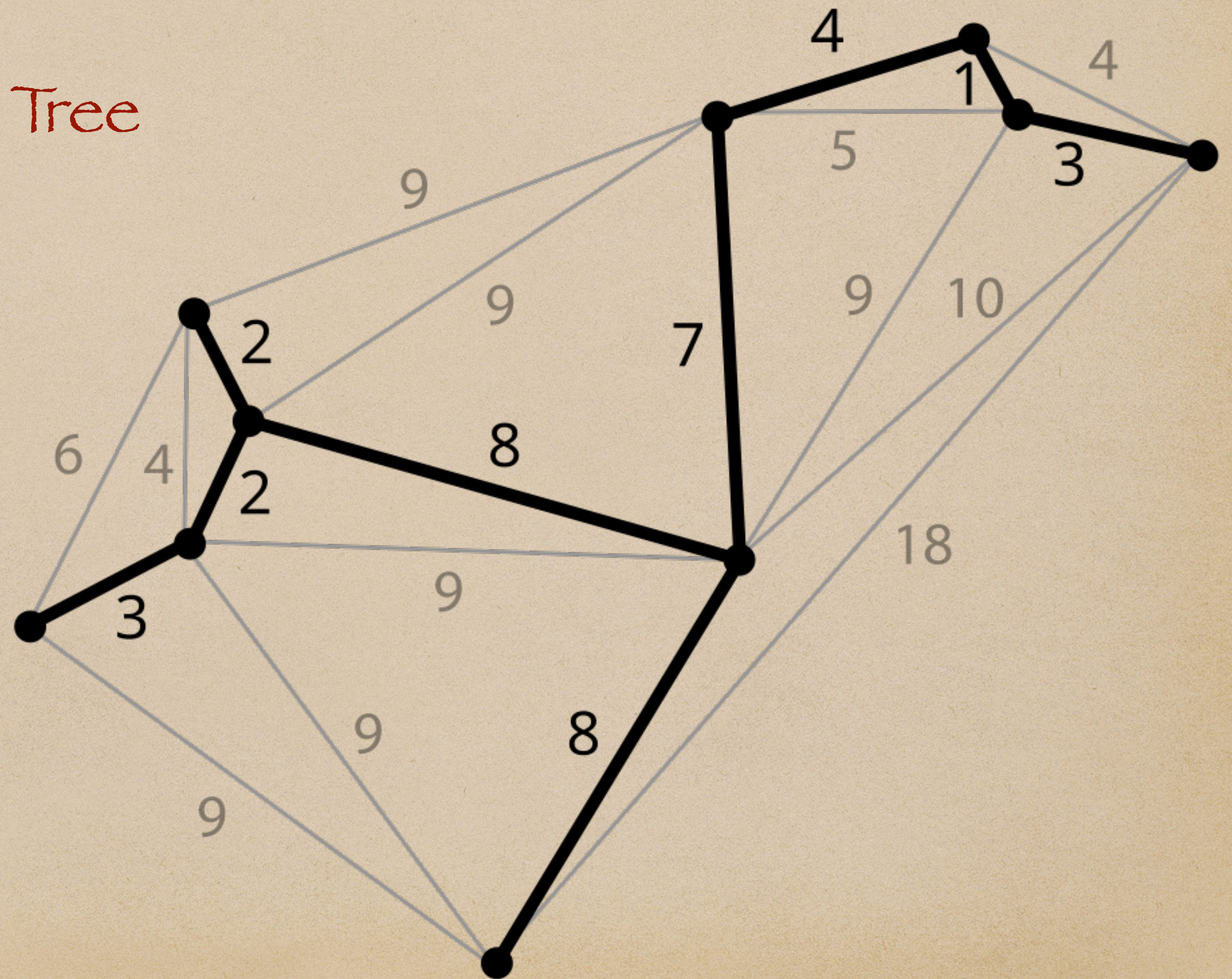
Recall Greedy

- ◆ Locally optimal, near-sighted, immediate benefit
- ◆ Sometimes intuitively clear, sometimes not!
- ◆ Correctness is often non-obvious; hence, concrete argument is required.

Recall Greedy

Classic Example: **Minimum Spanning Tree**

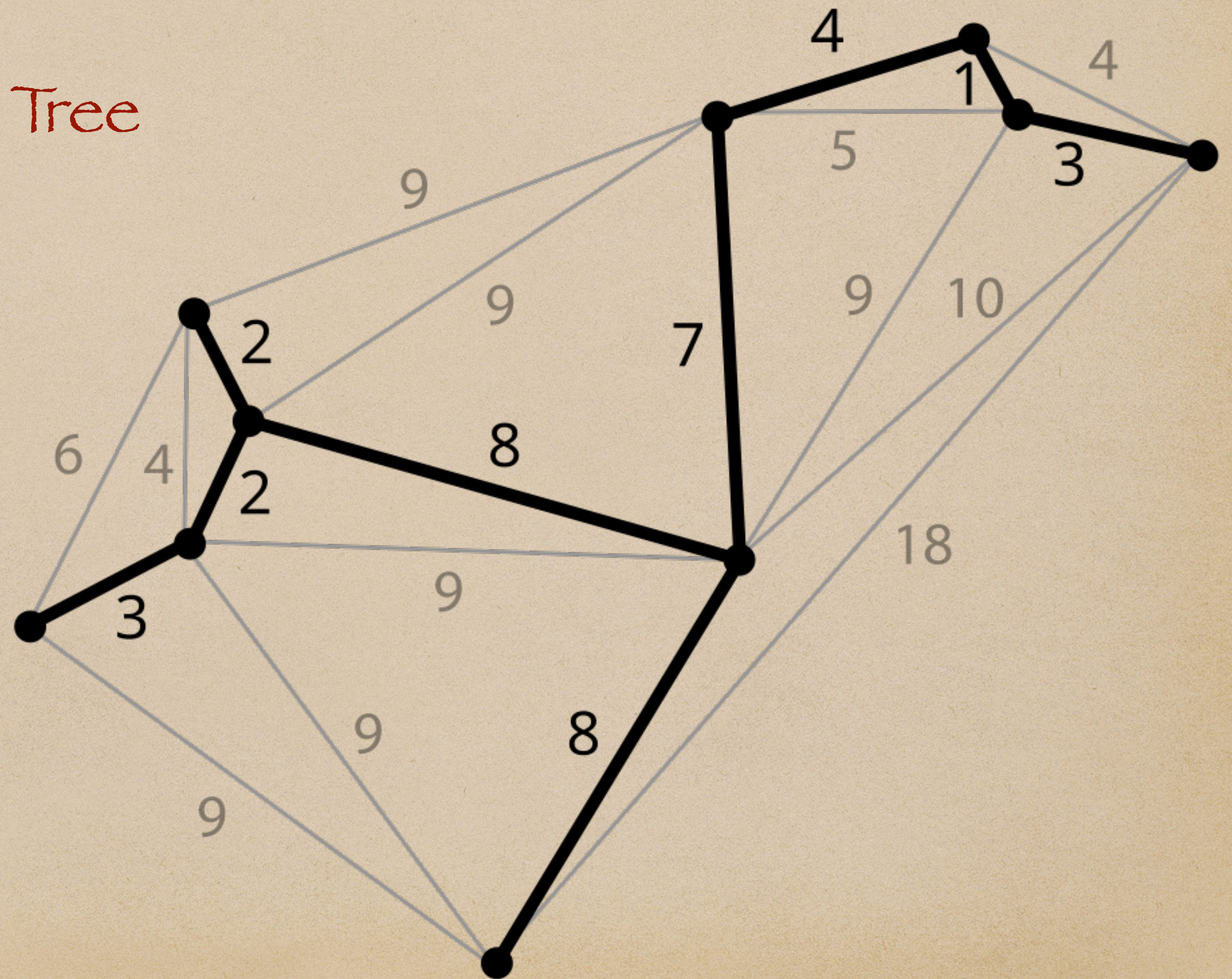
a subset of the edges of
a **connected**, edge-weighted
undirected **graph** that connects
all the **vertices** together, without
any **cycles** and with the minimum
possible total edge weight.



Recall Greedy

Classic Example: Minimum Spanning Tree

Property: Lexicographically
chooses the edges

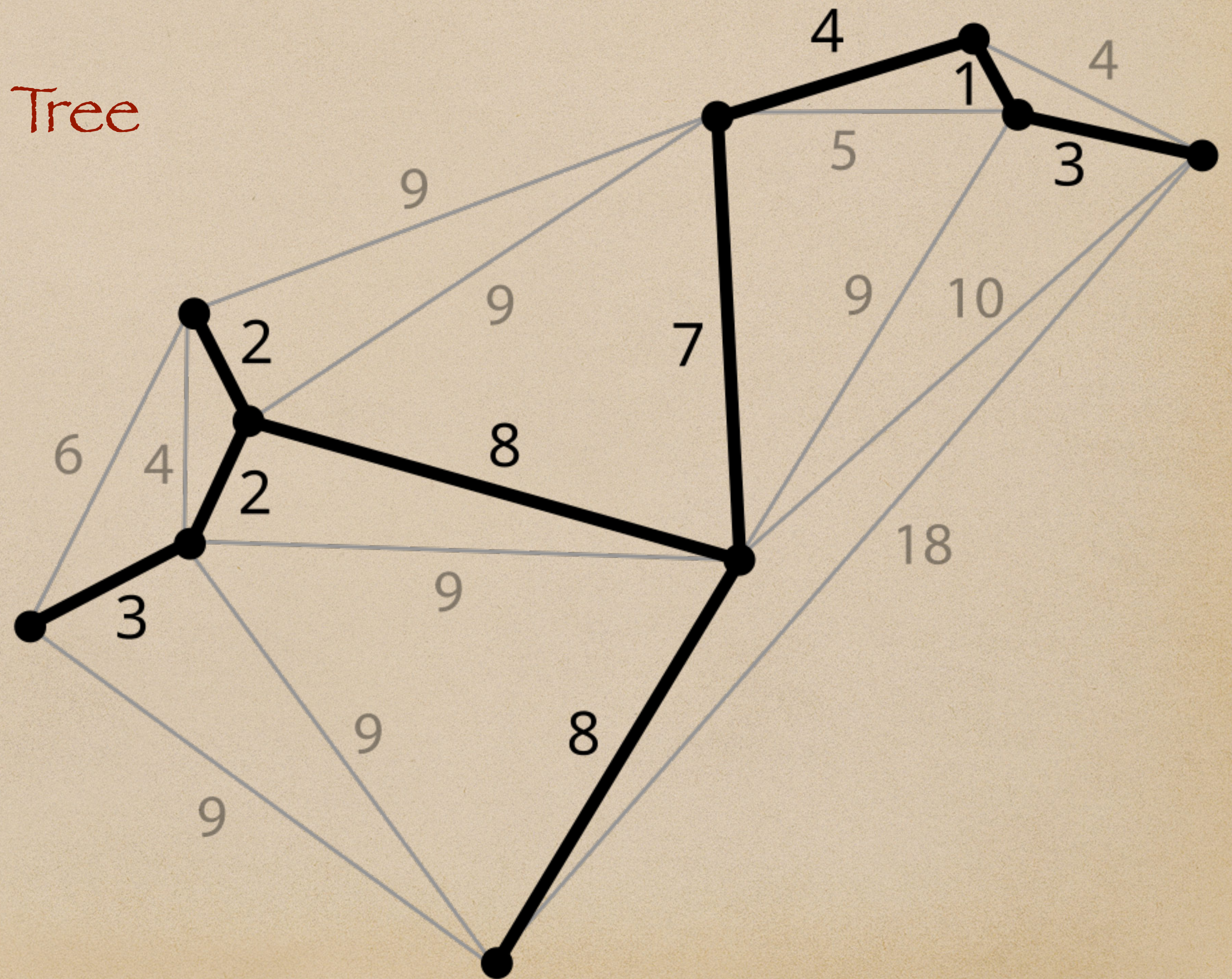


Recall Greedy

Classic Example: Minimum Spanning Tree

Property: Lexicographically
chooses the edges

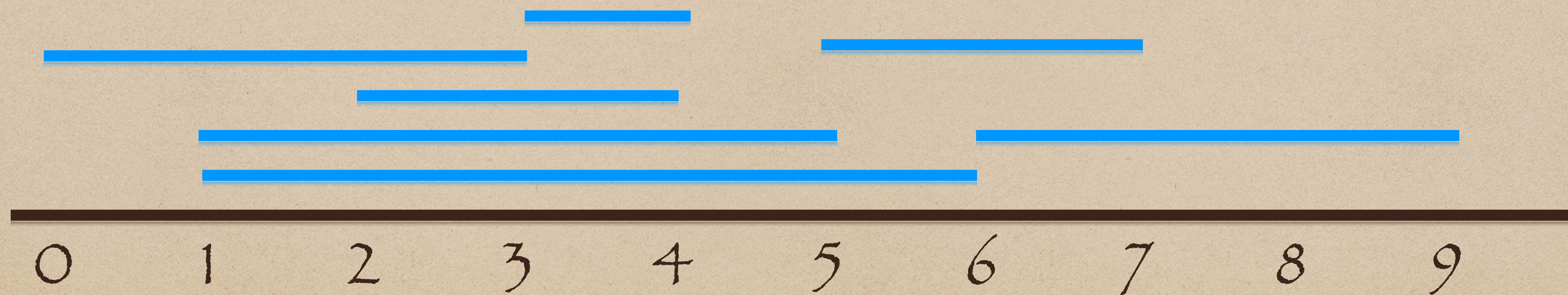
How to prove this?



Back to Interval Scheduling

[Kleinberg, Tardos; Chap. 4]

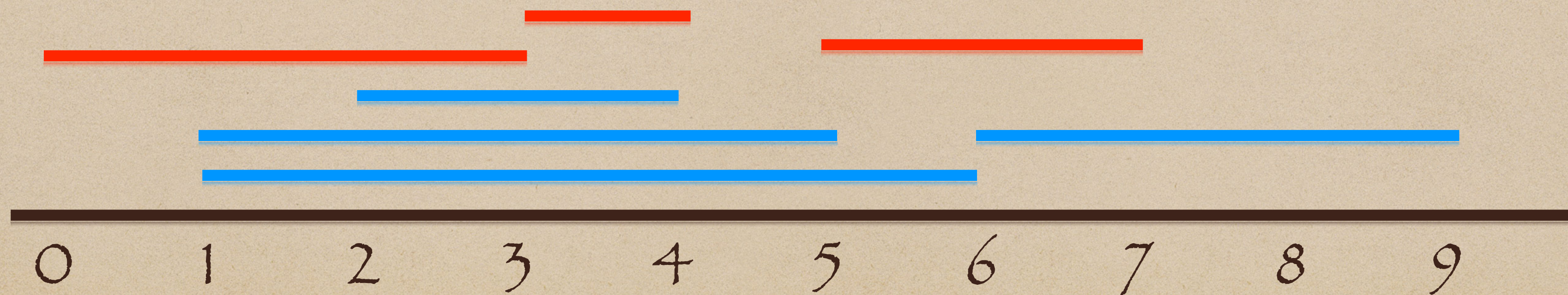
- ◆ Given a set of intervals on a real line, find the **largest** subset of **disjoint** intervals.
- ◆ Ex. $(1, 5)$, $(0, 3)$, $(2, 4)$, $(1, 6)$, $(3, 4)$, $(5, 7)$, $(6, 9)$.



Back to Interval Scheduling

[Kleinberg, Tardos; Chap. 4]

- ◆ Given a set of intervals on a real line, find the **largest** subset of **disjoint** intervals.
- ◆ Ex. $(1, 5)$, $(0, 3)$, $(2, 4)$, $(1, 6)$, $(3, 4)$, $(5, 7)$, $(6, 9)$.

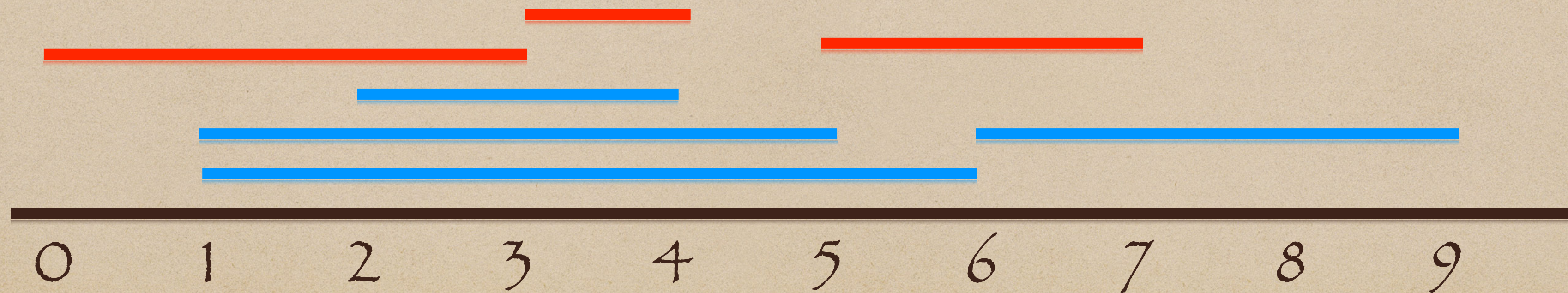


Back to Interval Scheduling

[Kleinberg, Tardos; Chap. 4]

Application: Resource allocation requests for fixed interval of time.

Searching through all possible solutions $\approx 2^n$ solutions!

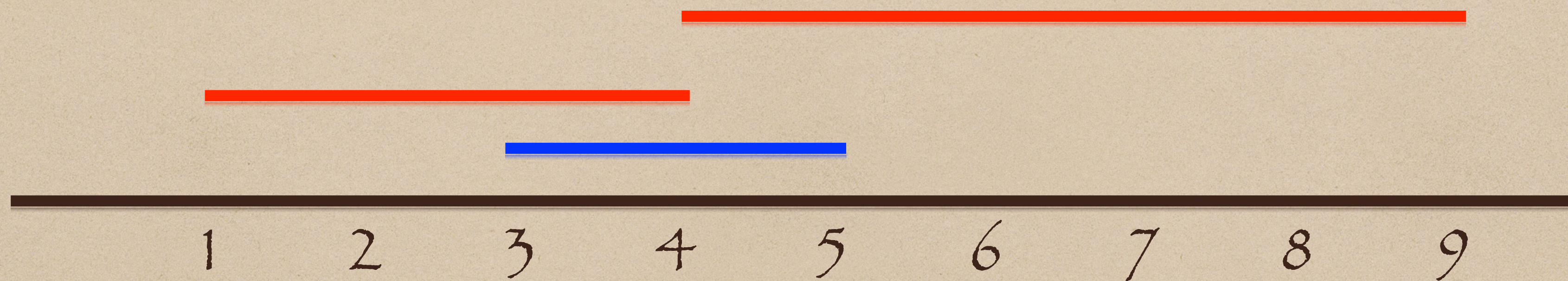


Interval Scheduling: Greedy Strategy - 1

- ◆ Keep choosing the smallest intervals, and remove the intervals that intersect with it.

Interval Scheduling: Greedy Strategy - I

- ◆ Keep choosing the smallest intervals, and remove the intervals that intersect with it.

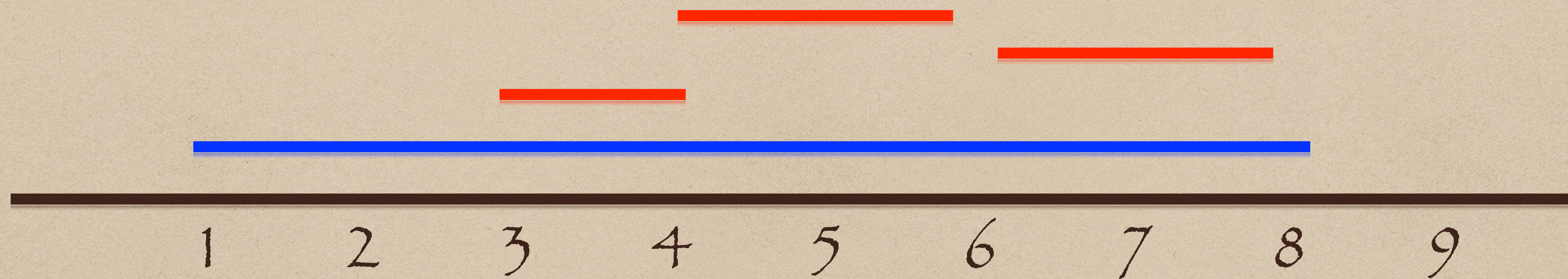


Interval Scheduling: Greedy Strategy - II

- ◆ Choose interval with earliest starting time...

Interval Scheduling: Greedy Strategy - II

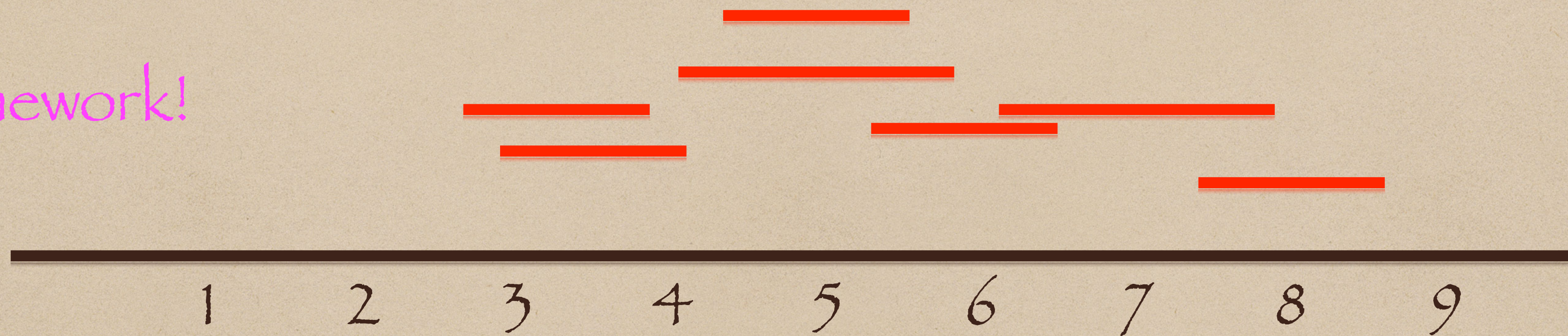
- ◆ Choose interval with earliest starting time...



Interval Scheduling: Greedy Strategy - III

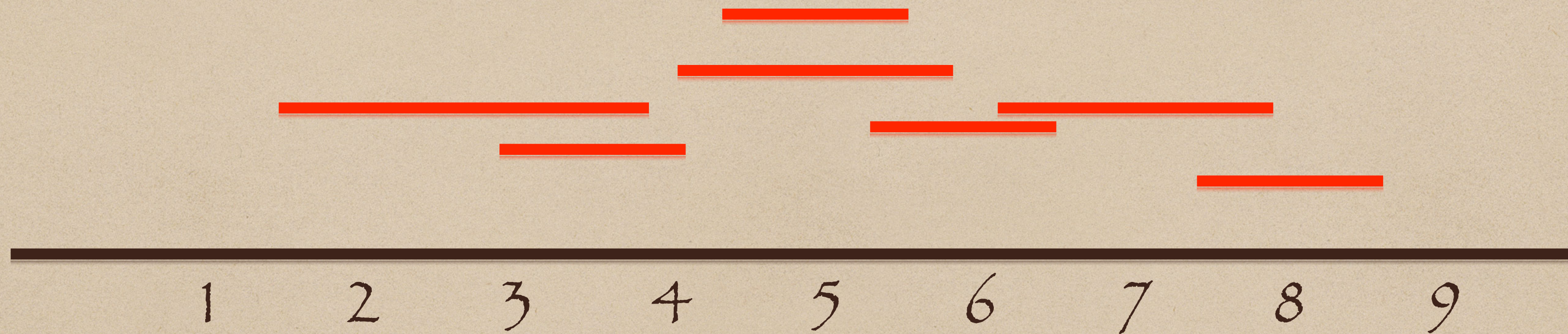
- ◆ Keep choosing the intervals with smallest number of overlaps.

Homework!



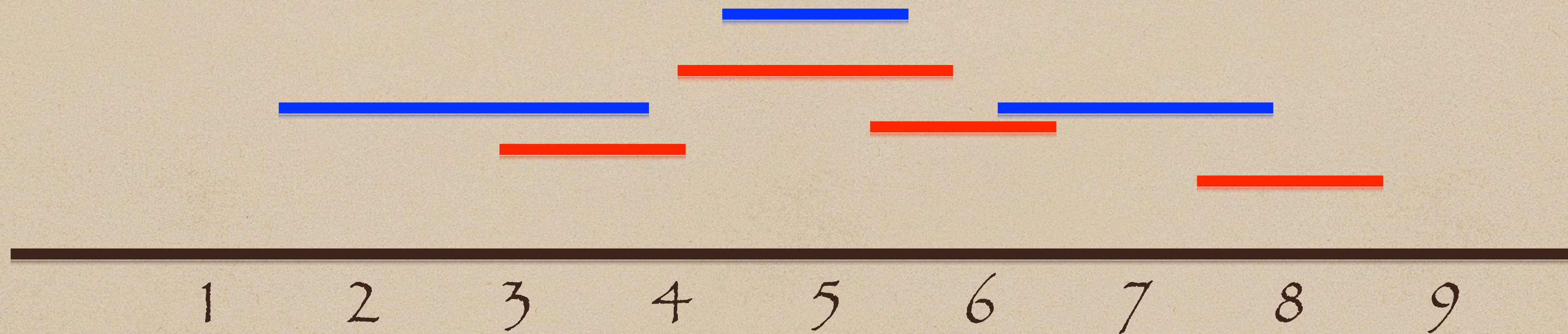
Interval Scheduling: Greedy Strategy - IV

- ◆ Keep choosing the intervals with earliest ending time.



Interval Scheduling: Greedy Strategy - IV

- ◆ Keep choosing the intervals with earliest ending time.



Intuition

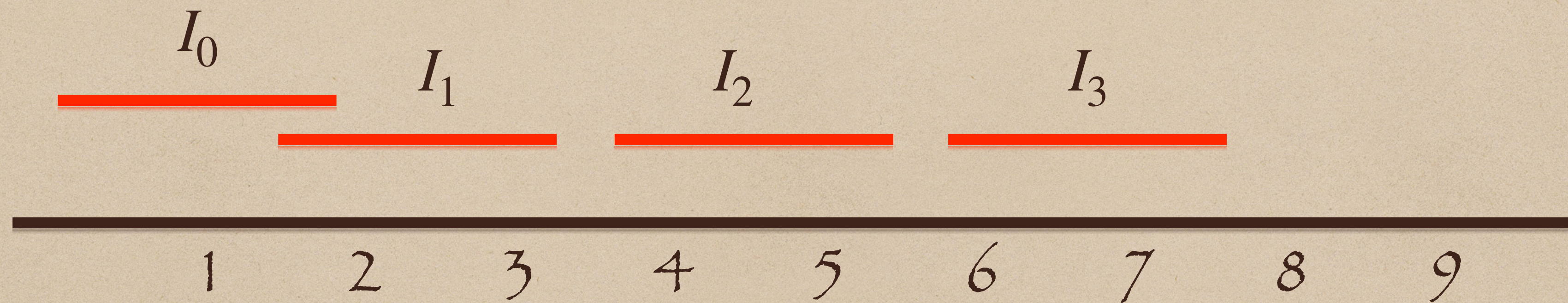
- ◆ We are maximising the remaining time, thus, maximising the possible number of request served afterwards!
- ◆ Claim: Greedy Strategy - IV gives an optimal solution.

Correctness

- ◆ Claim: Greedy Strategy - IV gives an optimal solution.
- ◆ Proof idea:
 - ◆ Show that there exists an optimal solution that “agrees” with the first greedy step.
 - ◆ The rest of the argument will work inductively.

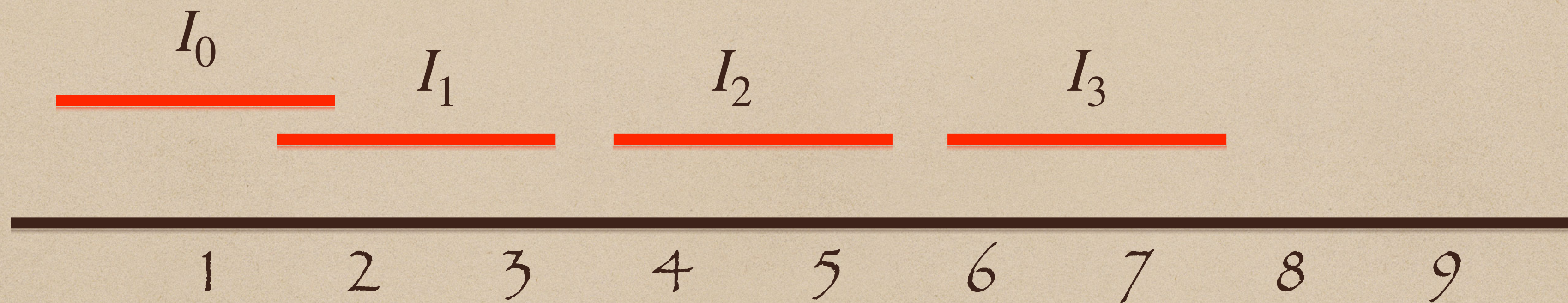
Correctness

- ◆ Consider an optimal solution $\sim I_1, I_1, I_2, \dots, I_k$
- ◆ But, earliest ending interval is I_0



Correctness

- ◆ But, earliest ending interval is I_0
- ◆ Swap I_1 with I_0 ... repeat



Correctness

- ◆ Claim: It's still a valid solution.
- ◆ Endtime (I_0) $\leq$ Endtime (I_1) $\leq$ Start time (I_2)
- ◆ I_0 doesn't overlap with remaining intervals, since I_1 didn't.

Hence, $I_0, I_2, \dots, I_k$ is an optimal solution that agrees with
The greedy first step.

Correctness - Induction!

- ◆ Inductive proof based on the number of intervals.
- ◆ Induction hypothesis: Greedy algorithm - IV works for any instance of upto $n - 1$ intervals.
- ◆ Inductive Step: Works for n th interval.

Correctness - Induction!

- ◆ Base case: $n = 1$ - obvious
- ◆ Algorithm - IV chooses I_1 , and removes all intervals that overlap with it.
- ◆ $\mathcal{J}' = \mathcal{J} - \{\text{Intervals overlapping with } I_1\}$.
- ◆ By the induction hypothesis we know, $\text{ALG}(\mathcal{J}') = \text{OPT}(\mathcal{J})$

Correctness - Induction

- ◆ Output - $I_1 + \text{ALG}(\mathcal{J}') = \text{OPT}(\mathcal{J}) + I_1$
- ◆ Claim: $(I_1 + \text{Any optimal solution for } \mathcal{J}')$ is an optimal solution for $\mathcal{J}'$.
 - ◆ Proof: Recall that, we showed that there is an optimal solution which includes I_1 . Say, $J_1, J_2, \dots, J_k$.
 - ◆ Hence, $\{J_1, J_2, \dots, J_k\}$ is a valid solution for $\mathcal{J}'$

Correctness - Induction

- ◆ Output - $I_1 + \text{ALG}(\mathcal{J}') = \text{OPT}(\mathcal{J}) + I_1$
- ◆ Claim: (I_1 + Any optimal solution for $\mathcal{J}'$) is an optimal solution for $\mathcal{J}'$.
- ◆ $|\text{OPT}(\mathcal{J}')| \geq |\{J_1, \dots, J_k\}|$
- ◆ $|\text{OPT}(\mathcal{J}') + I_1| \geq |\{I_1, J_1, \dots, J_k\}| = |\text{OPT}(\mathcal{J})| \dots$